

EstateFlow AI

Project Proposal: Multi-Tenant AI Real-Estate Automation Platform

Prepared for: Asif

Date: September 19, 2026

Executive Summary

EstateFlow AI is a production-oriented platform that automates real-estate customer service and operations. It combines a web application, Python backend services, retrieval-augmented generation (RAG), LangGraph agents, n8n workflows, voice and email automation, property management, appointments, human approval, observability, and security controls.

The platform will allow customers to ask property questions through web, email, voice, and messaging channels. It will retrieve relevant and current property information, generate grounded responses, qualify leads, check availability, schedule viewings, and escalate sensitive actions to staff. Agency employees will manage properties, documents, conversations, approvals, and analytics through a dashboard.

Project Purpose

The purpose of EstateFlow AI is to demonstrate how reliable AI automation can reduce manual real-estate work while preserving human control over important decisions. The project is designed as a complete portfolio and production-learning system for mastering AI automation engineering.

Problem Statement

Real-estate agencies often manage property information, customer inquiries, email, appointments, and follow-ups across disconnected tools. Staff must repeatedly search listings, answer similar questions, enter data into CRMs, schedule viewings, and route leads manually. This creates slow response times, inconsistent answers, missed follow-ups, and limited visibility into operations.

EstateFlow AI addresses this problem by connecting property knowledge, AI reasoning, business rules, and external applications into one controlled workflow platform.

Objectives

- Provide customers with fast, natural-language property search and support.
- Ground AI responses in verified property records and documents.
- Automate lead intake, classification, routing, follow-up, and notifications.
- Support voice, email, web, and messaging channels.
- Allow staff to approve sensitive AI-proposed actions.
- Maintain complete conversation, workflow, and audit histories.
- Measure AI quality, latency, cost, failures, and business outcomes.
- Demonstrate production-level skills across frontend, backend, AI, automation, security, and deployment.

Target Users

User	Needs
Customers	Search properties, ask questions, upload requirements, request viewings, and receive updates.
Sales agents	View leads, manage conversations, review AI drafts, and follow up with customers.
Managers	Approve sensitive actions, monitor team activity, and review appointments.
Administrators	Manage organizations, users, integrations, model usage, and system health.

Functional Scope

Module	Description
Customer portal	Authentication, chat, saved properties, appointments, profile, multilingual support, and file upload.
AI property assistant	Natural-language intent detection, property retrieval, availability checks, grounded responses, and citations.
Staff dashboard	Lead management, conversations, properties, documents, approvals, appointments, workflows, and analytics.
Property management	Create, update, publish, archive, and search property listings with images and documents.
Document intelligence	Extract, clean, chunk, embed, index, update, and archive property documents.
Voice automation	Receive voice events, store transcripts, summarize calls, qualify leads, and trigger follow-ups.
Email automation	Classify inbound email, create leads, draft replies, route messages, and trigger escalation.
Appointment automation	Check availability, create approval requests, schedule calendar events, and send reminders.
Human approval	Pause risky actions for review, editing, approval, rejection, and safe resumption.
Observability	Track traces, model calls, tool calls, latency, token use, cost, quality, and errors.

Technical Architecture

The proposed architecture separates user experience, business APIs, AI orchestration, data storage, and external automation. This makes the system easier to test, maintain, scale, and explain in technical interviews.

Frontend: Next.js, TypeScript, Tailwind CSS, shadcn/ui, React Query, React Hook Form, Zod, and Recharts.

Backend: Python, FastAPI, Pydantic, SQLAlchemy, Alembic, PostgreSQL, Redis, and background workers.

AI layer: OpenAI/Anthropic, LiteLLM, LangChain, LangGraph, Pinecone, and LangSmith or Langfuse.

Automation: n8n for webhooks, email, CRM, calendar, Slack, approvals, scheduled workflows, and error handling.

Infrastructure: Docker, Docker Compose, HTTPS reverse proxy, GitHub Actions, cloud deployment, backups, and monitoring.

Core flow:

Customer or business event → n8n or web frontend → FastAPI validation → LangGraph agent → RAG or business tools → response/action validation → approval if required → external action → audit log and monitoring.

AI Agent Design

The agent will be implemented as a controlled state graph instead of an unrestricted chatbot. It will classify requests, retrieve information, call approved tools, validate evidence, and pause for human review when an action has operational or financial risk.

Example graph:

START → load context → classify intent → retrieve properties or call a business tool → validate evidence → generate response/action → perform risk check → deliver response or request approval → save audit event → END.

Potential tools include property search, property details, availability checks, appointment requests, approved email sending, CRM updates, staff-task creation, and human escalation. Each tool will validate arguments, enforce permissions, apply tenant filters, record execution details, and handle timeouts.

LangGraph persistence will support conversation continuity and safe pause/resume behavior for human approval. [web:87][web:88]

RAG Pipeline

Property data will be indexed through a controlled ingestion pipeline: upload or synchronize a source → validate and store the file → extract text and metadata → split content into chunks → create embeddings → store vectors in Pinecone → save document version and indexing status in PostgreSQL.

Retrieval will include query understanding, metadata filters, vector search, optional hybrid retrieval, reranking, context assembly, source references, and verification against current PostgreSQL availability data. Each vector will include organization and property metadata to enforce tenant isolation.

User Interface Plan

- Public landing page with search, featured properties, AI assistant call-to-action, and agency information.
- Customer dashboard with recent conversations, recommendations, saved properties, and appointments.
- Chat interface with streaming responses, property cards, citations, voice input, upload, and escalation.
- Staff dashboard with lead metrics, open conversations, approvals, appointments, and failed workflows.
- Property management interface with listing forms, images, documents, publishing status, and indexing status.
- Approval inbox with proposed action, evidence, risk flags, edit, approve, reject, and escalate controls.
- Analytics interface showing response time, cost, quality, escalation rate, and conversion metrics.
- Administration interface for users, organizations, roles, integrations, AI settings, and usage limits.

Database Scope

Initial tables will include organizations, users, organization members, roles, customers, properties, property images, documents, conversations, messages, agent runs, agent actions, tool calls, approvals, appointments, integrations, workflow events, audit logs, usage records, evaluation cases, and evaluation runs.

All organization-owned data will contain an organization identifier. Backend queries and Pinecone metadata filters will enforce tenant isolation so that one agency cannot access another agency's customers, documents, or properties.

Implementation Plan

Phase	Deliverables
Phase 1: Foundation	Create repositories, Docker setup, database, authentication, organization roles, FastAPI health endpoint, and Next.js dashboard shell.
Phase 2: Property management	Implement property CRUD, images, documents, publishing status, search, and staff-facing forms.
Phase 3: RAG MVP	Implement document upload, extraction, chunking, embeddings, Pinecone indexing, metadata filters, and property search.
Phase 4: Customer chat	Implement conversations, messages, streaming responses, property cards, citations, and chat history.
Phase 5: LangGraph agent	Add intent routing, tools, structured state, persistence, validation, and controlled responses.
Phase 6: Approvals and appointments	Add availability checks, appointment requests, human approval, calendar integration, confirmations, and reminders.
Phase 7: n8n automation	Add email intake, CRM updates, Slack notifications, follow-ups, webhook security, retries, and error workflows.
Phase 8: Voice and multilingual support	Connect ElevenLabs, store transcripts, summarize calls, support Urdu/English routing, and escalate low-confidence cases.
Phase 9: Production operations	Add LiteLLM, observability, evaluations, security tests, CI/CD, queue workers, backups, load tests, and deployment.
Phase 10: Portfolio delivery	Write technical documentation, create architecture diagrams, record a demo, publish metrics, and prepare an interview presentation.

Minimum Viable Product

The first working release should include only the smallest complete vertical slice: staff login, property creation, document upload, RAG indexing, customer chat, property retrieval, cited answer display, and conversation storage. This prevents the project from becoming an unfinished collection of integrations.

MVP success flow:

Staff logs in → adds properties → uploads property data → documents are indexed → customer asks a question → the agent retrieves matching properties → the UI displays a grounded response with sources → the conversation is saved.

Testing and Evaluation

The system will be tested at four levels: backend unit and integration tests, frontend component and user-flow tests, n8n workflow failure tests, and AI evaluation tests. The evaluation dataset should contain realistic property questions, ambiguous requests, missing-information cases, prompt-injection attempts, cross-tenant access attempts, and unsafe action requests.

Key metrics include retrieval precision and recall, answer correctness, citation accuracy, intent classification accuracy, tool-call success rate, average and P95 latency, cost per conversation, escalation rate, and workflow failure rate. AI evaluation should be run whenever prompts, models, retrieval logic, or agent behavior changes. [web:2][web:4]

Security and Reliability

- Use environment variables or secret management; never commit API keys.
- Verify webhook signatures and validate all external input.
- Enforce authentication, role-based authorization, and tenant isolation.
- Validate LLM outputs and tool arguments with schemas.

- Use read-only access where a tool only needs retrieval.
- Require human approval for booking, discounts, bulk messages, and other sensitive actions.
- Implement idempotency, retries, timeouts, and duplicate-event prevention.
- Record audit logs for AI decisions, tool calls, approvals, and external actions.
- Mask sensitive data in logs and maintain encrypted backups.
- Add monitoring, alerts, dependency updates, and recovery procedures.

Expected Outcomes

- A deployable web application for AI-supported real-estate operations.
- A reusable Python and FastAPI foundation for LLM applications.
- Practical mastery of LangChain, LangGraph, RAG, vector databases, tool calling, and human-in-the-loop design.
- Production-level n8n skills including webhooks, sub-workflows, approvals, retries, monitoring, and scaling concepts.
- A portfolio project demonstrating frontend, backend, AI, automation, security, DevOps, and product thinking.
- Measurable evidence of response time, automation rate, AI quality, cost, and reliability.

Portfolio and Career Value

This project positions the developer as an AI Automation Engineer specializing in Python, LLM applications, RAG, AI agents, n8n orchestration, API integrations, and production deployment. It directly demonstrates the ability to build business systems rather than isolated chatbot demonstrations.

The final portfolio should include a public architecture diagram, UI screenshots, a short product demonstration, API documentation, database design, agent graph, n8n workflow screenshots, evaluation results, security decisions, deployment instructions, and a case study describing measurable business outcomes.

Definition of Done

- The platform accepts inquiries from at least three channels.
- The AI assistant retrieves current property information with citations.
- Availability and appointment actions are validated against business data.
- Sensitive actions pause for human approval and resume safely.
- Duplicate events and common API failures are handled safely.
- Tenant data is isolated across the database and vector store.
- AI quality, latency, cost, and failures are observable.
- Automated tests and evaluation cases run in CI/CD.
- The system is containerized and deployed to a staging or production environment.
- The project has complete documentation and a recorded demonstration.

Conclusion

EstateFlow AI is intentionally designed as a large, staged project. The first milestone should be a working RAG property assistant with a clean UI and saved conversations. After that foundation is stable, add LangGraph tools, approvals, n8n integrations, voice, analytics, observability, security, and deployment. This sequence provides continuous working results while covering the complete AI Automation Engineer skill set.

Prepared as a project proposal and implementation blueprint.